

The Coversheet

Student Name (unless anonymised)	
Student Number (as shown on student ID card):	
Title	A Critical Analysis of Cybersecurity Assessment and Penetration Testing Techniques
Word Count / Pages / Duration / Other Limits:	
Attempt Number:	
Date of Submission:	

I have read and understood the Academic Misconduct statement .	Tick to confirm ☐
I have read and understood the Generative Artificial Intelligence use statement .	Tick to confirm ☐
I am satisfied that I have achieved the Learning Outcomes of this assignment (please check the Assignment Brief if you are unsure)	Met ☐

Self-Assessment – If there are particular aspects of your assignment on which you would like feedback, please indicate below.
Optional for students

Suggested prompt questions-

How have you developed or progressed your learning in this work?
What do you feel is the strongest part of this submission?
What feedback would you give yourself?
What part(s) of this assignment are you still unsure about?

Assessor's Feedback (may be delivered in line with the submission)

Were the learning outcomes met?	Yes ☐ If not, what was not met:
Assessor's response to the student's submission, request for feedback and / or self-assessment (feedback):	
What specific actions should the student undertake to progress their learning? (feedforward):	
Please take this and other feedback to your next academic tutorial to plan your future work.	

List of Figures

Introduction

Cybersecurity assessments are crucial for detecting vulnerabilities, evaluating security measures, and minimising risks to organisations. Organisations can use ethical hacking, vulnerability assessment, encryption, reconnaissance, and security testing techniques to understand potential attack paths before malicious actors exploit them. This report critically evaluates key concepts in cybersecurity, including footprinting, DNS enumeration, vulnerability management, privilege escalation, fuzzing, port scanning, encryption technologies, and ethical hacking methodologies. Throughout, the relevance of these techniques is illustrated through academic theory, industry practice, and real-world examples.

Part 1: Footprinting and DNS Enumeration

1. Penetration Testers Use Footprinting to Build a Comprehensive Profile of a Target

a. Concept and Importance of Footprinting

Footprinting is the initial and critical phase in the whole process of ethical hacking and penetration testing. As per the experts, footprinting is considered a process that involves gathering information on a particular computer system, network, or enterprise before performing an attack. The primary objective behind footprinting is obtaining complete information on the target concerning their network infrastructure, operating system, services, and security status. Besides this, it is imperative to realise that this process holds great significance. In case the preliminary reconnaissance did not take place appropriately, penetration testers will overlook certain avenues for performing the attack, waste time targeting inappropriate places and attract undue attention from their opponents. From the attackers' point of view, footprinting offers several benefits as well.

Footprinting can be generally classified into two categories: passive footprinting, where information is collected without actually interacting with the system of interest, and active footprinting, where interaction with the system is involved. Both approaches are important parts of the OSINT (Open-Source Intelligence) approach popular today among security specialists. While footprinting is a vital part of penetration testing, it is highly dependent on publicly available information. Organisations of the modern world are increasingly using different services aimed at protecting their privacy and using cloud and security services that prevent them from leaking

information. Nevertheless, there is plenty of useful intelligence that can still be obtained through social networks, online storage, and other sources.

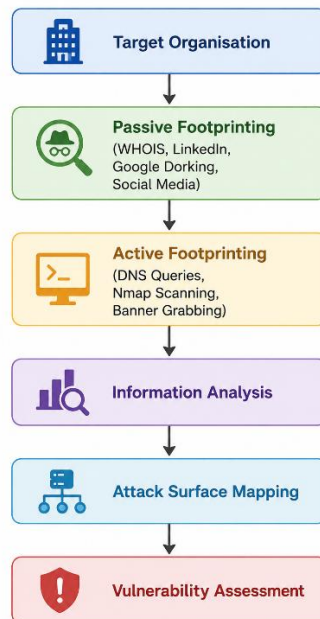


Figure 1: Footprinting Process in Penetration Testing

b. Technical and Non-Technical Methods Used in Footprinting

Technical Methods:

- **WHOIS Lookup:** Information contained in WHOIS databases includes registration data on domain names and IP addresses, such as owner, email address, domain servers, and date of registration. Some useful tools used by attackers include whois.domaintools.com or Linux `whois` to obtain such information. An example of such a query is 'whois example.com,' which reveals the domain's registrar, IP addresses (CIDR format), and contacts.
- **DNS Interrogation:** Querying for DNS information may help to find the IP addresses (A), email servers (MX), name servers (NS), and text verification (TXT) that will disclose more information about the infrastructure. DNS zone transfers, which occur through an AXFR query, will give you a list of all the hostnames in a certain domain if performed correctly.
- **Google Dorking (Advanced Search Operators):** By employing specialised search queries using Google (dorking), one is able to identify exposed configuration files, login forms, errors, and directories. For instance, `site:example.com filetype: pdf confidential` identifies any publicly indexed PDF files that contain the word confidential.

- **Network Scanning with Ping Sweeps and Traceroute:** Software like Nmap helps one identify which hosts are up and what the path to the target host looks like in terms of intermediate routers and firewalls.

Non-Technical Methods:

- **Social Engineering and OSINT:** The analysis of the organisation's social media (LinkedIn, Twitter, Facebook) will help to get information about employees, positions, technology being used, and organisational structure. The LinkedIn profiles usually disclose the technologies by listing skills and job descriptions.
- **Job Advertisements:** The job descriptions may actually reveal the software version, security measures, and network structure used by the business. The job description that states 'Cisco ASA 5500-X' is used shows the firewall technology being used.
- **Physical Reconnaissance:** Observe the physical locations of the organisation, perform Dumpster Diving to obtain documents with passwords, network maps, or server names, and tailgate into the physical facility by following authorised employees.
- **Website and Source Code Analysis:** The website belonging to the target itself may reveal details about its technology stack using HTTP headers (Server, X-Powered-By), HTML comments made by the developers, and meta tags. Software like BuiltWith or Wappalyzer performs this job.

c. Defensive Measures Against Unauthorised Footprinting

Organisations can implement a layered set of controls to significantly reduce the information available to external reconnaissance:

- **WHOIS Privacy Protection:** Using privacy services during the domain name registration process to ensure that the domain information is provided anonymously by replacing it with proxy details. This would make sure that no organisation or employee data is revealed using a WHOIS lookup.
- **DNS Hardening:** Blocking DNS zone transfers (AXFR) for public authoritative DNS servers by restricting access to only legitimate secondary DNS servers. By enabling DNS Security Extensions (DNSSEC), the DNS responses gain cryptographic integrity checks, eliminating cache poisoning vulnerabilities and increasing security.
- **Web Application Firewalls (WAFs) and HTTP Security Headers:** Eliminating the Server and X-Powered-By HTTP header fields from response data to hinder technology

identification. Installing a WAF screens out suspicious activity like automated web crawling and dorking by search engines like Google.

- **Information Security Awareness Training:** Educating employees to avoid disclosing sensitive technical details on social media, professional networks, and job advertisements. Policies should mandate the review of job postings to remove technology-specific details.
- **Network Traffic Monitoring and Threat Intelligence:** Using SIEM technologies for the detection and alerts regarding anomalies related to large numbers of DNS requests, port scanning activities, and Shodan crawlers, thereby providing the opportunity for prompt defensive measures.
- **Digital Footprint Audits:** such as searches through search engines, social media platforms, and third-party information aggregation services in order to remove any sensitive information (ISO/IEC 27001:2022).
- **Email Security Controls:** Proper configuration of SPF, DKIM, and DMARC records for preventing email spoofing and reducing the potential of intelligence gathering via email headers analysis; proper mail relay configuration for removal of internal IP headers.

2. Methods of DNS Enumeration and Standard Tools Used During DNS Enumeration

DNS Enumeration - Methods and Tools

DNS Enumeration is a technique wherein one exhaustively investigates the domain name structure of the target organisation in order to gather all the hostnames, IP addresses, and email servers that may be connected to this organisation. This activity is very important since it is a vital component of footprinting due to the public nature of the DNS structure.

Methods of DNS Enumeration

- **DNS Zone Transfer (AXFR):** A malicious individual or hacker tries to acquire a complete copy of the DNS zone by making a request for such data from the authoritative name server. When a name server does not require proper authentication before performing a zone transfer, the result will include all the records in the zone, basically giving away all host names within the organisation.
- **Sub-domain Brute Force Enumeration:** Sub-domain brute force enumeration uses wordlists of typical subdomains (mail, vpn, dev, staging, api) along with DNS resolution to find out the hosts that do not exist on any publicly available information.

- **Reverse DNS Lookups (PTR Records):** A reverse DNS lookup makes use of DNS queries based on IP addresses in order to obtain host names and identify patterns in server naming schemes.

Tool 1: dnsenum

dnsenum is a DNS enumerator coded in the Perl programming language. This tool is often included in penetration testing suites such as Kali Linux. DNSenum conducts zone transfers, dictionary attacks to enumerate subdomains, and queries on Google to obtain more hosts, in addition to performing reverse DNS lookups on the discovered IP address ranges.

Practical Example:

```
dnsenum --enum -f /usr/share/dnsenum/dns.txt -o output.xml  
targetdomain.com
```

Tool 2: Fierce

Fierce is a DNS reconnaissance tool that is especially meant for identifying the non-contiguous IP space and hostnames from defined domains. Unlike normal DNS lookup tools, Fierce will try doing zone transfers first and then do brute-force enumeration using its internal wordlist. In addition, Fierce will do reverse lookups on the identified subnets to determine adjacent hosts in neighbouring IP space that do not belong to the target domain (Security experts at Bishop Fox created Fierce).

Practical Example:

```
fierce --domain targetdomain.com --subdomains wordlist.txt --delay 1
```

According to the perspective of penetration testing, brute-forcing remains the most effective technique for enumerating subdomains because many companies fail to develop, test, and maintain legacy systems properly. However, the effectiveness of domain name system reconnaissance is hindered by some improvements such as DNSSEC, zone transfer control, and cloud DNS security. Regardless of the improvement in the DNS system, poorly maintained and forgotten subdomains continue leaking sensitive information.

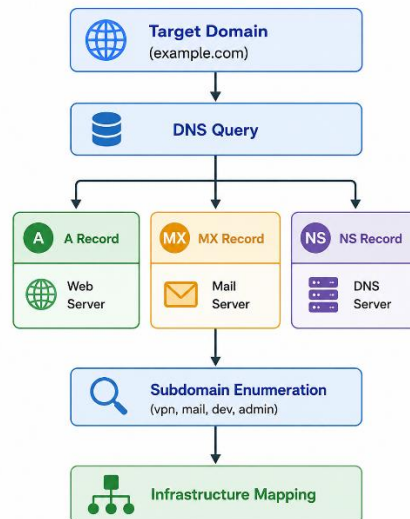


Figure 2: DNS Enumeration Process and Infrastructure Mapping

Part 2: CVE System, Vulnerability Scanning, Privilege Escalation, and Fuzzing

1. The Common Vulnerabilities and Exposures (CVE) System

What is the CVE System?

Common Vulnerabilities and Exposures provide an approach for describing vulnerabilities in the computer industry using unique names. The CVE names provide consistency when conveying information about security vulnerabilities in software. CVEs are used during penetration tests to determine the existence of known vulnerabilities, exploit these vulnerabilities, and prioritise the necessary remediation efforts.

CVE Identifiers

The CVE Identifier or CVE ID is a unique alphanumeric string that is associated with a particular vulnerability. It has a standard format: CVE-[YEAR]-[NUMBER]. The [YEAR] component refers to the year in which the vulnerability was identified or assigned, whereas the [NUMBER] component corresponds to a sequential number. For instance, the CVE-2021-44228 is the CVE identifier associated with the Log4Shell vulnerability in Apache Log4j, disclosed in December 2021. Each CVE identifier includes the following: a standardised description of the vulnerability, details on software versions impacted, relevant information from vendors' advisories, CVSS base score (v3.1 or v4.0), temporal and environmental metrics, along with the status of the CVE entry (Reserved, Published or Rejected). Moreover, the National Vulnerability Database (NVD) also

provides CWE classification of the root cause vulnerability, for instance, SQL Injection (CWE-89) and XSS Vulnerabilities (CWE-79).

Table 1: Critical Common Vulnerabilities and Exposures (CVEs) Used for Vulnerability Assessment

CVE Component	Description
CVE-2021-44228	Apache Log4j Remote Code Execution (Log4Shell) – CVSS 10.0 Critical
CVE-2017-0144	MS17-010 EternalBlue – SMB Remote Code Execution used by WannaCry ransomware
CVE-2019-19781	Citrix ADC Path Traversal leading to Remote Code Execution
CVE-2022-0847	Dirty Pipe – Linux Kernel Privilege Escalation

Referring to CVE Identifiers During a Security Test

During a penetration test, CVE identifiers are referenced at multiple stages of the assessment lifecycle. The primary occasions include:

- **Vulnerability Scanning and Identification:** Scanning tools such as Nessus, OpenVAS, and Qualys will match identified services and software versions against the CVE database and return a set of relevant CVE IDs with the severity scores. This helps the tester in deciding which vulnerabilities can be attacked according to the CVSS score and publicly available exploits.
- **Exploitation Phase:** If Metasploit Framework is used, the respective exploit module is normally referred to by CVE ID (such as exploit/windows/smb/ms17_010_eternalblue refers to CVE-2017-0144). With the CVE in mind, the tester will check whether the found version of the service is vulnerable and then launch an exploitation attempt.
- **Post-Exploitation and Lateral Movement:** Local software installed on the compromised server can be checked for CVE identifiers for privilege escalation possibilities, as, for instance, with kernel exploits listed under CVE-2022-0847 (Dirty Pipe).
- **Report Writing and Remediation Guidance:** All the vulnerabilities found during the penetration test should be listed together with respective CVEs if necessary. This will allow the client organisation to look into vendor advisories, determine the availability of the patch, and evaluate its severity using the CVSS score. The use of CVE ID removes any ambiguity.

2. A Well-Known Automated Vulnerability Scanner and the Merits of Vulnerability Identification

Vulnerability Scanning - Manual vs Automated Approaches

One such example of an automated vulnerability scanner is Nessus, which is manufactured by Tenable Inc. It is among the most popular and widely used commercial scanners across the globe and is able to scan networks, websites, clouds, and IoTs for hundreds of vulnerabilities that are registered in the CVE and NVD databases.

Merits of Manual Vulnerability Identification

- **Context-Aware Assessment:** A knowledgeable penetration tester can spot logical flaws, business logic problems, and vulnerability chains that cannot be detected by automated tools. For example, an IDOR vulnerability could potentially be discovered by a manual tester because of their familiarity with the context of the application under test.
- **Lower False Positives:** The main issue with automated scanning is that they produce false positives. A manual check determines if the flagged vulnerability can actually be exploited, providing better reports.
- **Zero-Day Exploits and New Vulnerabilities:** A penetration tester conducting a manual assessment may discover vulnerabilities without CVE numbers, which adds value not found in automated databases.

Merits of Automated Vulnerability Scanning (Nessus)

- **Speed and Coverage:** Automated scanning tools can test hundreds of machines at once, performing thousands of vulnerability tests within a fraction of the time it takes humans to perform the same task. With a single scan, Nessus can perform more than 100,000 plugin tests, guaranteeing full coverage of all vulnerabilities on a network.
- **Consistency and Repeatability:** Scanning tools ensure that each test is performed consistently for all targets. Scheduled scans ensure continuous testing and help organisations stay compliant with regulations like PCI-DSS and ISO/IEC 27001 that mandate regular scanning.
- **CVE and Patch Management Integration:** Since Nessus refers to the CVE ID, CVSS scores, and patches available, integration with patch management processes becomes seamless. The dashboard gives high-level executives information about risks while providing detailed recommendations for resolving issues.

However, an efficient test involves the use of both methodologies. The automation of vulnerability testing ensures fast processing and efficiency of results, but on-site tests ensure accuracy and

identification of chain attacks. Neither the manual nor the automated approaches to vulnerability assessment can be sufficient on their own. Automation is associated with speed, efficiency, and consistency of results; manual tests reveal weaknesses in business logic as well as sophisticated attack paths that usually elude automatic systems. As a result, companies using only scanners may have a wrong picture of their vulnerability profile.

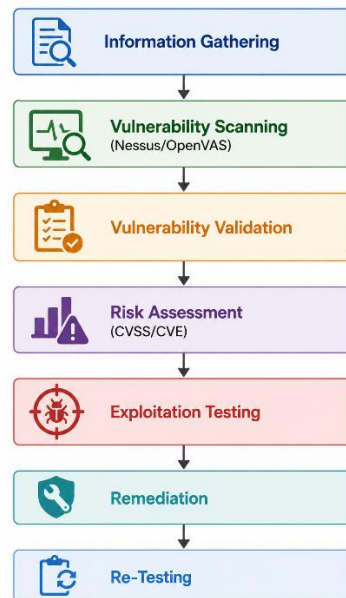


Figure 3: Vulnerability Assessment Lifecycle

3. Techniques for Obtaining Higher Privileges on a Compromised Host

Privilege Escalation Techniques

Once the penetration tester gains access to the host through an account with minimal privileges, they will have to escalate their privileges in order to take full control of the system and move laterally within the network.

Technique 1: Exploiting Misconfigured SUID/SGID Binaries (Linux)

In the Linux operating system, the term SUID stands for Set User ID, which is a unique file attribute which makes an executable run under the permissions of the owner and not the invoking user. In case there is any flaw within a SUID file which is owned by the root account, it could be exploited to gain full root privileges.

Practical Example: The find command with SUID permissions can be exploited as follows:

```
find / -perm -u=s -type f 2>/dev/null  
find . -exec /bin/sh -p \; -quit
```

Technique 2: Windows Token Impersonation (Potato Attacks)

The attacks performed against privilege escalation in the Windows environment usually exploit the weakness that lies within the Windows token and impersonation mechanism. Among such attacks are a group of attacks known as the Potato attacks, which comprise Hot Potato, Rotten Potato, Juicy Potato, and Sweet Potato attacks. These attacks utilise the Windows COM/DCOM server and NTLM Relay attack method to impersonate access tokens at the SYSTEM level through a low-privileged service account. Some of the most successful targets for these attacks include service accounts that run using IIS_IUSRS, LOCAL SERVICE, and NETWORK SERVICE on compromised web servers.

Practical Example using Juicy Potato:

```
JuicyPotato.exe -l 9999 -p C:\Windows\System32\cmd.exe -a "/c whoami > C:\output.txt" -t *
```

Another technique that complements Juicy Potato, especially in an environment where it has been patched, is PrintSpoofer, which utilises the Windows Print Spooler service to impersonate SYSTEM tokens in the same way.

4. The Testing Method Known as Fuzzing

Fuzzing as a Security Assessment Method

Fuzz testing is an automated software testing method whereby random inputs are provided to the target program. These can either be malformed, unanticipated inputs or data generated from scratch. The tester looks out for program crashes, memory corruption, unanticipated behaviour, or any unhandled exception. Fuzz testing plays an important role in current-day security testing since it has led to the identification of many CVEs in well-known applications. Fuzzing can be classified by the level of knowledge the fuzzer has about the target:

Table 2: Types of Security Testing Based on Knowledge of System Internals

Type	Description
Black Box	No knowledge of source code
White Box	Full knowledge of source code
Grey Box	Partial knowledge of the system

Fuzzing is widely used for assessing web applications, API security, operating systems, and network vulnerabilities through detecting crashes, memory corruption and input validation vulnerabilities. The key advantage of using fuzzing techniques in security audits performed by organisations is the capability to discover zero-day vulnerabilities that will be invisible to signature detection and manual code review. Empirical proof of security flaws can be obtained through discovering reproducible crashes. This makes fuzzing especially appealing in terms of presenting results of penetration testing performed against development teams. Fuzzing becomes increasingly popular in the age of CI/CD by being integrated into them through fuzzing platforms like OSS-Fuzz developed by Google. However, fuzzing cannot replace other security testing methods. For example, random generation of inputs cannot reveal vulnerabilities related to the need to perform certain business actions or the authentication of users.

Part 3: Port Scanning and Encryption

1. Port Scanning and Port States

Port Scanning – Method, Example, and Port States

Port scanning is one form of reconnaissance that can be used to discover active ports and services that are running on the target machine. A port refers to an endpoint that facilitates communication within the TCP/IP protocol, and these ports range from 0 to 65535, while the first 1024 are standardised ports (such as TCP port 80 for HTTP, TCP port 443 for HTTPS, TCP port 22 for SSH). The most widely used port scanning tool is Nmap (Network Mapper), an open-source utility developed by Gordon Lyon. A typical SYN scan (half-open scan) command is:

```
nmap -sS -sV -O -p 1-1024 --open 192.168.1.100

22/tcp    open      ssh       OpenSSH 8.4p1 Debian
80/tcp    open      http      Apache httpd 2.4.46
443/tcp   open      https     Apache httpd 2.4.46
3306/tcp   filtered  mysql
```

The Three Port States

Table 3: Comparative Analysis of Open, Closed, and Filtered Port States

Port State	Explanation
Open	There is an active listening application or service that receives TCP connections or UDP datagrams on this port. There is a listening service

	configured for use on the open port. An open port means that an attack vector has been identified. On an Nmap SYN scan, an open port returns the SYN-ACK response when the SYN packet is received by the targeted host.
Closed	The port can be reached; however, there is no service listening on it. On an Nmap SYN scan, the port sends an RST packet response. Closed ports remain relevant since they can help with determining if the host machine is alive, assist in determining the OS used based on TCP/IP stack fingerprinting, and become open if a service is running in the future.
Filtered	Firewall, router ACLs, or other devices in the path of the probe are interfering with the process; hence, the status of the port is uncertain as to whether it is open or closed. This state is usually accompanied by either no response or an ICMP unreachable error message. Filtered ports hinder the tester from learning about the topology of the network.

2. Implementing Encryption to Prevent Unauthorised Access to Open Ports

Implementing Encryption to Protect Open Ports

When port scanning identifies open ports on a target system, the primary encryption-based countermeasures an organisation can implement to prevent unauthorised access include:

- **Implementation of TLS/SSL for Application Services:** Applications using unencrypted open ports such as HTTP (TCP 80), SMTP (TCP 25), and FTP (TCP 21) will be switched to their corresponding encrypted counterparts like HTTPS (TCP 443), SMTPS (TCP 587/465), and FTPS/SFTP (TCP 990/22), respectively, using TLS certificates. The current best practice recommendation is the use of TLS version 1.3 (RFC 8446), which uses ephemeral Diffie-Hellman key exchange, providing forward secrecy along with elimination of outdated cypher suites susceptible to BEAST, POODLE, and SWEET32 attacks (IETF, 2018). TLS certificate issued by the trusted authority (internal CA) authenticates the server, ensuring clients' confidentiality against man-in-the-middle interception of unencrypted traffic over these ports.
- **Enforcement of VPNs and Whitelisting Open Ports:** Open ports found during a scan will be secured by enforcing VPN connectivity. Technologies like WireGuard, OpenVPN, and IPsec/IKEv2 provide encryption and authentication for all client-to-server communication. Together with a firewall (iptables or cloud security group) and whitelisting

of only authorised IPs belonging to the encrypted VPN tunnel, this ensures that all open ports will be invisible to outside scans.

3. Common Encryption Algorithms Used in Modern Cybersecurity Practices

Comparison of Encryption Algorithms in Modern Cybersecurity

Two encryption algorithms that form the basis of contemporary cybersecurity include the Advanced Encryption Standard and the Rivest-Shamir-Adleman algorithm. The former refers to the prevalent approach to symmetric encryption, while the latter is synonymous with asymmetric cryptography.

Table 4: Comparative Analysis of AES and RSA Cryptographic Algorithms

Feature	AES (Symmetric)	
Feature	AES	RSA
Type	Symmetric block cypher	Asymmetric public-key cypher
Key Sizes	128, 192, or 256 bits	1024, 2048, 4096 bits (2048+ recommended)
Algorithm Base	Substitution-permutation network (SPN)	Integer factorisation problem
Performance	Extremely fast; hardware-accelerated (AES-NI)	Computationally expensive; ~1000x slower than AES
Key Distribution	The shared secret key must be securely exchanged	Public key freely shared; private key kept secret
Primary Use	Bulk data encryption (files, disks, TLS payload)	Key exchange, digital signatures, certificate authentication
Security (2024)	AES-256 is considered quantum-resistant for the near term	Vulnerable to Shor's algorithm on quantum computers
Standards	FIPS 197, ISO/IEC 18033-3	PKCS#1, RFC 8017, X.509 certificates

AES - Advanced Encryption Standard

The Advanced Encryption Standard (AES) is a symmetric block cypher standard that was developed by the National Institute of Standards and Technology (NIST) in 2001 (FIPS 197). AES operates on fixed-length data blocks of 128 bits with a possible key size of either 128, 192, or 256 bits. AES uses multiple rounds of encryption to protect information from theft or attacks and is

currently used to encrypt bulk data since it is the industry standard in encryption. The new AES-256 in Galois/Counter Mode is now considered the gold standard for authenticated encryption with associated data (AEAD), which guarantees both confidentiality and integrity in one go. It is used by TLS 1.3, IPsec, and WPA3 wireless protection, as well as full-disk encryption, BitLocker, and FileVault. The latest CPUs provide instruction sets known as AES-NI (Advanced Encryption Standard New Instructions), which deliver throughput of more than 10 GB/s. Therefore, AES encryption is almost computationally negligible for enterprises today.

RSA – Rivest-Shamir-Adleman

RSA cryptosystem is a public-key asymmetric encryption scheme based on the problem of factoring a large semi-prime number $n=pq$, where p and q are large prime numbers. The public key (n, e) is calculated using these prime numbers and publicly distributed, while the private key (n, d) is secretly stored. Encryption is performed using the public key, and decryption requires the private key. The strength of the RSA-2048 encryption lies in the inability to factorise a 617-digit number even if decades of calculations are performed on classical computers. RSA is predominantly used for key exchange, digital signatures, and certificate authorities. RSA key exchange is deprecated in TLS 1.3 in favour of ephemeral Elliptic Curve Diffie-Hellman for forward secrecy, whereas, in certificate authentication, RSA is widely used. One crucial flaw in RSA's security is that RSA is not secure in a post-quantum world due to Shor's algorithm, which performs polynomial-time prime factorisation in quantum computers of sufficient computing power. NIST's post-quantum cryptography standardisation (FIPS 203-205) effort aims at creating post-quantum counterparts of RSA algorithms like lattice-based CRYSTALS-Kyber and CRYSTALS-Dilithium.

In modern-day cybersecurity, AES and RSA work in tandem in the context of hybrid cryptography approaches. In such cases, RSA (or ECDH) is used as the asymmetric approach to secure the transfer of AES session keys, which will be responsible for encrypting large amounts of data efficiently. This type of system capitalises on both the advantages offered by asymmetrical encryption in the form of key exchanges and symmetric encryption in terms of fast bulk data encryption. This method is well illustrated by the TLS handshake protocol defined by the IETF (RFC 8446). Despite being extensively used for signature and verification processes, RSA faces substantial threats due to quantum computing developments.

Part 4: Ethical Hacking

1. Different Steps of Ethical Hacking and the Aim of Each Step



Figure 4: Ethical Hacking Execution Process

Steps of Ethical Hacking and Their Aims

Ethical Hacking, also referred to as penetration testing or offensive security, employs a process-oriented approach to evaluate the security position of any organisation. The EC Council CEH and Penetration Testing Execution Standard (PTES) describe five basic phases that have particular aims, just as malicious hackers do, but in an ethical, legally authorised environment.

Table 5: The Five Phases of the Ethical Hacking and Penetration Testing Methodology

Phase	Description and Aim
1. Reconnaissance (Footprinting)	The aim is to extract as much information regarding the target as possible, utilising both passive and active measures without any involvement with the target system. It will involve methods such as OSINT, DNS enumeration, WHOIS lookups, social media research, and network mapping. The result of this phase will be a detailed report about the target system, which is later used as an input for the next steps.
2. Scanning and Enumeration	The aim is to discover active hosts, open ports, services that are running, and versions of the software through active probing of the target's

	network border. The use of such tools as Nmap and Nessus will be involved. Enumeration is a step beyond scanning and involves discovering additional information such as user accounts, shares on the network, service banners, and configurations from discovered services (for instance, SNMP enumeration).
3. Gaining Access (Exploitation)	This phase aims at exploiting the discovered vulnerabilities to gain access to systems that should otherwise remain inaccessible. Such techniques involve the use of tools like Metasploit, custom-written exploit code, and social engineering. The exploits can be aimed against unpatched software (vulnerabilities listed by CVE numbers), incorrect configurations, use of default credentials, or logical flaws. The purpose of this step is to show practical consequences, such as shell setup, credential harvesting, and sensitive information collection.
4. Maintaining Access (Post-Exploitation)	This phase focuses on testing the organisation's ability to detect and respond to incidents caused by the first stage of the attack. The steps include establishing backdoors, creating privileged user accounts, setting up Command and Control (C2) communication channels, and moving laterally through the network in order to target other accessible systems within the network. The goal of this step is not only to see how vulnerable the system was, but also how good the organisation's detection and response capability is.
5. Covering Tracks and Reporting	In terms of an attacker's approach, the point here is to remove traces of the intrusion. This can include wiping the logs of the server, eliminating malware artefacts, and reversing any changes that have been made during the penetration test. From the ethical hacker's point of view, this phase is equivalent to report writing, when findings and evidence are documented.

Despite having a set methodology, ethical hacking has become increasingly complicated due to the advancement of cloud platforms and integration through the hybrid infrastructure, which involves collaboration with other third parties. This calls for constant modification of the existing methodologies by the hackers.

2. Real-World Ethical Challenges Faced by Ethical Hackers

Ethical Challenge 1: Scope Creep and Unintended System Compromise

In the process of performing penetration testing, it is quite common for penetration testers to find themselves facing systems that are within the close physical/logical proximity of the authorised system but not covered under the ROE document. An ethical dilemma for penetration testers in authorised testing arises when one detects a serious vulnerability in a third-party supply chain system that is outside the authorised scope but connects with the authorised part of the system. The ethical hacker is obligated by the Rules of Engagement to stop exploiting the vulnerability beyond the scope line; however, if the vulnerability pathway remains undisclosed to the client organisation, it leaves the firm vulnerable to attack.

This dilemma places the tester in a difficult situation regarding contract obligations, ethical responsibility, and even legality under the Computer Misuse Act of 1990. The ethical approach in such a case would be to record details of the newly found path, stop any further action towards the system which is out of scope, and mention an 'Out of Scope Observation' section in the report that recommends conducting an assessment of the new system. According to the EC Council Code of Ethics, "testers should not access, use, or attempt to circumvent controls on systems outside the agreed scope."

Ethical Challenge 2: Discovery of Evidence of Active Criminal Activity

However, in the post-exploitation and data exfiltration phase of simulation, penetration testing personnel can unintentionally come across evidence of actual criminal conduct on the targeted computer system. Such evidence could include any type of content, such as CSAM or the organisation conducting crimes against its clients. The question of what action a penetration tester should take in such a situation is a highly delicate issue since, in their contractual terms, the testers are bound to keep all information about vulnerabilities confidential and report only the security flaws of their client. On the other hand, it may be the requirement of professional ethics or even criminal legislation to report this information to authorities.

The ethical approach would involve discontinuing further access to any additional material discovered (in order to prevent tainting evidence), contacting one's legal advisor within the organisation, and, in instances where there has been severe harm like CSAM, contacting the appropriate authorities (NCA in the UK, FBI in the US) despite any contractual agreement with the client. In fact, the CREST (Council of Registered Ethical Security Testers) Code of Conduct addresses such an instance, saying that one must comply with all laws and report crimes despite any contractual confidentiality agreement.

Conclusion

This report covered various cybersecurity aspects in relation to reconnaissance, vulnerability assessment, privilege escalation, encryption, and ethical hacking. It has been shown how methods such as footprinting, DNS enumeration, vulnerability scanning, fuzzing, and port scanning can be used in the identification of security flaws within an organisation's environment. Additionally, it is worth noting that effective measures have been recommended for the implementation of a good security strategy. These include the application of defensive mechanisms, encryption and ethical principles in performing the security audit process.

References